

"Divide and Conquer & Decrease and Conquer"

- Q and A

BIET College, Dept. of AIML

April 20, 2026

Contents

1	Question: Explain the concept of divide and conquer. Design an algorithm for merge sort and derive its time complexity. (10 Marks)	3
2	Question: Design Quicksort Algorithm and Apply it to Sort: 25, 75, 40, 10, 20, 05, 15. Derive Its Efficiency. (10 Marks)	6
3	Question: Explain Strassen's matrix multiplication and derive its time complexity. (10 Marks)	10
4	Question: Write an algorithm to sort numbers using Insertion Sort. Discuss its efficiency. (10 Marks)	13
5	Question: Design Quicksort Algorithm and Analyze Its Efficiency (10 Marks)	15
6	Question: Write Merge Sort algorithm and sort the list E, X, A, M, P, L, E. (8 Marks)	18
7	Question: Apply the DFS-based algorithm to solve the Topological Sorting problem. (6 Marks)	21
8	Question: Write algorithms for Preorder, Inorder and Postorder traversals of a tree. Write traversals for the given tree. (6 Marks)	23
9	Question: What is Divide and Conquer? Develop Quick Sort algorithm and write its best case. Use it to sort: E, X, A, M, P, L, E. (10 Marks)	26
10	Question: Distinguish between Decrease & Conquer and Divide & Conquer with block diagram. Develop Insertion Sort algorithm and estimate efficiency. (10 Marks)	29
11	Question: Define Topological Sorting. List the two approaches and illustrate with examples. (10 Marks)	32

- 12 Question: Apply Quick Sort to sort: P, R, O, G, R, A, M, M, I, N, G.
Draw the tree of recursive calls. (10 Marks) 34
- 13 Question: Apply Merge Sort and Quick Sort on: V, T, U, B, E, L,
A, G, A, V, I. Draw tree of recursive calls and merge operations. (10
Marks) 36
- 14 Question: Apply Strassen's Algorithm for matrix multiplication and
justify why it is better. (10 Marks) 39
- 15 Question: Write the control abstraction for Divide and Conquer tech-
nique. (4 Marks) 41
- 16 Question: Divide and Conquer (4M); Stable Algorithms & Quicksort
(6M); Sorting by Position vs Value (10M) 43

1 Question: Explain the concept of divide and conquer. Design an algorithm for merge sort and derive its time complexity. (10 Marks)

Introduction

The **divide-and-conquer** technique is a fundamental algorithmic strategy that solves complex problems by breaking them into smaller subproblems, solving them recursively, and combining their results.

Concept of Divide-and-Conquer

The approach follows three key steps:

1. **Divide:** The given problem is partitioned into **smaller subproblems** of the same type, ideally of equal size.
2. **Conquer:** The subproblems are **solved recursively**. If a subproblem is small enough, it is solved directly (base case).
3. **Combine:** The solutions of the subproblems are **merged** to obtain the solution to the original problem.

Note: **Mergesort** is a classic example where division is simple and most work happens during merging.

Design of Merge Sort Algorithm

Merge Sort consists of two main procedures:

1. Recursive Procedure: Mergesort($A[0..n-1]$)

Input: Array $A[0..n-1]$

Output: Sorted array in non-decreasing order

Algorithm 1 Mergesort($A[0..n-1]$)

```

1: //Sorts array  $A[0..n-1]$  by recursive mergesort
2: //Input: An array  $A[0..n-1]$  of orderable elements
3: //Output: Array  $A[0..n-1]$  sorted in nondecreasing order
4: if  $n > 1$  then
5:   copy  $A[0..\lfloor n/2 \rfloor - 1]$  to  $B[0..\lfloor n/2 \rfloor - 1]$ 
6:   copy  $A[\lfloor n/2 \rfloor..n-1]$  to  $C[0..\lfloor n/2 \rfloor - 1]$ 
7:   Mergesort( $B[0..\lfloor n/2 \rfloor - 1]$ )
8:   Mergesort( $C[0..\lfloor n/2 \rfloor - 1]$ )
9:   Merge( $B, C, A$ )
10: end if

```

2. Merge Procedure: Merge(B, C, A)

Steps:

1. Initialize three pointers: $i = 0, j = 0, k = 0$
2. Compare elements of $B[i]$ and $C[j]$:
 - Copy the **smaller element** into $A[k]$
 - Increment corresponding indices
3. Repeat until one array is exhausted.
4. **Copy remaining elements** from the non-empty array into A .

Algorithm 2 Merge($B[0..p-1], C[0..q-1], A[0..p+q-1]$)

```

1: // Merges two sorted arrays into one sorted array
2: // Input: Arrays  $B[0..p-1]$  and  $C[0..q-1]$  both sorted
3: // Output: Sorted array  $A[0..p+q-1]$  of the elements of  $B$  and  $C$ 
4:  $i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$ 
5: while  $i < p$  and  $j < q$  do
6:   if  $B[i] \leq C[j]$  then
7:      $A[k] \leftarrow B[i]; i \leftarrow i + 1$ 
8:   else
9:      $A[k] \leftarrow C[j]; j \leftarrow j + 1$ 
10:  end if
11:   $k \leftarrow k + 1$ 
12: end while
13: if  $i = p$  then
14:   copy  $C[j..q-1]$  to  $A[k..p+q-1]$ 
15: else
16:   copy  $B[i..p-1]$  to  $A[k..p+q-1]$ 
17: end if

```

Time Complexity Derivation

1. Recurrence Relation

Let $C(n)$ be the number of comparisons.

$$C(n) = 2C(n/2) + C_{\text{merge}}(n), \quad C(1) = 0$$

2. Merge Step Analysis

In the worst case, merging requires:

$$C_{\text{merge}}(n) = n - 1$$

3. Solving the Recurrence

$$C(n) = 2C(n/2) + n - 1$$

Using the **Master Theorem**:

- $a = 2, b = 2, f(n) = n$
- $n^{\log_b a} = n^{\log_2 2} = n$

Since $f(n) = \Theta(n^{\log_b a})$, Case 2 applies:

$$C(n) = \Theta(n \log n)$$

Exact solution (for $n = 2^k$):

$$C(n) = n \log_2 n - n + 1$$

Conclusion

Merge Sort achieves $\Theta(n \log n)$ time complexity in all cases (best, average, worst), making it highly efficient. However, it requires $\Theta(n)$ extra space for auxiliary arrays during merging.

2 Question: Design Quicksort Algorithm and Apply it to Sort: 25, 75, 40, 10, 20, 05, 15. Derive Its Efficiency. (10 Marks)

Part 1: Design of the Quicksort Algorithm (4 Marks)

Definition: Quicksort is a **divide-and-conquer** algorithm that sorts by partitioning an array around a **pivot** element. All elements smaller than the pivot are moved to its left, and all larger elements to its right. The pivot is then in its final sorted position, and the process is repeated recursively on the subarrays.

Pseudocode

Algorithm 3 Quicksort($A[l..r]$)

```

1: Input: Subarray of  $A[0..n-1]$ , defined by indices  $l$  and  $r$ 
2: Output: Subarray  $A[l..r]$  sorted in ascending order
3: if  $l < r$  then
4:    $s \leftarrow \text{PARTITION}(A[l..r])$  ▷  $s$  is split/pivot position
5:   QUICKSORT( $A[l..s-1]$ )
6:   QUICKSORT( $A[s+1..r]$ )
7: end if

```

Algorithm 4 Partition($A[l..r]$) (Hoare's method)

```

1: Input: Subarray  $A[l..r]$ 
2: Output: Final index of the pivot
3:  $p \leftarrow A[l]$  ▷ Pivot is the first element
4:  $i \leftarrow l$ 
5:  $j \leftarrow r + 1$ 
6: repeat
7:   repeat
8:      $i \leftarrow i + 1$ 
9:   until  $A[i] \geq p$ 
10:  repeat
11:     $j \leftarrow j - 1$ 
12:  until  $A[j] \leq p$ 
13:  SWAP( $A[i], A[j]$ )
14: until  $i \geq j$ 
15: SWAP( $A[i], A[j]$ ) ▷ Undo last swap where  $i$  and  $j$  crossed
16: SWAP( $A[l], A[j]$ ) ▷ Place pivot in its correct position
17: return  $j$ 

```

Part 2: Applying Quicksort to the List: 25, 75, 40, 10, 20, 05, 15 (6 Marks)

We will trace the algorithm step-by-step. The initial array is: $A = [25, 75, 40, 10, 20, 05, 15]$ (Indices 0 to 6)

Step 1: First Call to Partition (l=0, r=6)

- **Pivot (p) = 25**
- **i** starts at 0, **j** starts at 7.

Action	i moves right until $\geq$ 25	j moves left until $\leq$ 25	Array State After Action
Scan 1	i stops at 75 (index 1)	j stops at 15 (index 6)	Swap(75, 15) $\rightarrow$ [25, 15, 40, 10, 20, 05, 75]
Scan 2	i stops at 40 (index 2)	j stops at 05 (index 5)	Swap(40, 05) $\rightarrow$ [25, 15, 05, 10, 20, 40, 75]
Scan 3	i stops at 40 (index 5)	j stops at 20 (index 4)	i (5) > j (4) $\rightarrow$ Loop Ends

- **Crossing Detected:** $i \geq j$. Undo last swap (swap(40,20) *before* checking? The algorithm structure in pseudocode does the swap then checks. The trace corrects: the loop executes swap(40,20) before checking, but then **i** becomes 6, **j** becomes 3, $i > j$, so we undo swap(40,20) by swapping them back. Effectively no change in that last step.)
- **Final Swap:** Place Pivot. Swap $A[l]$ (25 at index 0) with $A[j]$ (20 at index 4).
- **Array becomes:** [20, 15, 05, 10, 25, 40, 75]
- **Split Position s = 4.** (Pivot 25 is fixed at index 4).

Step 2: Recursive Call on Left Subarray [20, 15, 05, 10] (l=0, r=3)

- **Pivot (p) = 20**
- **i = 0, j = 4 (r+1)**

Action	i moves right ($\geq$ 20)	j moves left ($\leq$ 20)	Result
Scan 1	i stops at 25 (index 4) – out of bounds, would stop at r; effectively 25 > pivot)	j stops at 10 (index 3)	i > j immediately. Loop ends.

- **Crossing Detected:** No swaps occurred.
- **Final Swap:** Swap $A[l]$ (20 at index 0) with $A[j]$ (10 at index 3).
- **Array becomes:** [10, 15, 05, 20, 25, 40, 75]
- **Split Position s = 3.**

Step 3: Recursive Call on Subarray [10, 15, 05] (l=0, r=2)

- **Pivot (p) = 10**
- **i = 0, j = 3**

Action	i moves right (≥ 10)	j moves left (≤ 10)	Array State
Scan 1	i stops at 15 (index 1)	j stops at 05 (index 2)	Swap(15, 05) $\rightarrow$ [10, 05, 15, 20, 25, 40, 75]
Scan 2	i moves to index 2 (15) and stops	j moves to index 1 (05) and stops	i (2) > j (1) $\rightarrow$ Loop Ends

- **Crossing Detected:** Undo last swap (swap back 05 and 15) $\rightarrow$ array goes back to [10, 15, 05, 20, 25, 40, 75].
- **Final Swap:** Swap pivot (10 at index 0) with $A[j]$ (05 at index 1).
- **Array becomes:** [05, 10, 15, 20, 25, 40, 75]
- **Split Position s = 1.**

Step 4: Remaining Subarrays

- **Left of 10:** [05] (single element, base case reached).
- **Right of 10:** [15] (single element, base case reached).
- **Right of 20:** None (index 4 onward already contains 25 which was first pivot's right subarray).

Step 5: Right Subarray of First Pivot [40, 75] (l=5, r=6)

- **Pivot (p) = 40**
- **i = 5, j = 7**
- i stops at 75 (index 6), j stops at 40 (index 5). $i > j$ immediately.
- **Final Swap:** Swap pivot (40 at index 5) with $A[j]$ (itself).
- **Array remains:** [..., 40, 75]
- **Split Position s = 5.**

Final Sorted Output

05, 10, 15, 20, 25, 40, 75

Summary of Splits (Tree Diagram View)

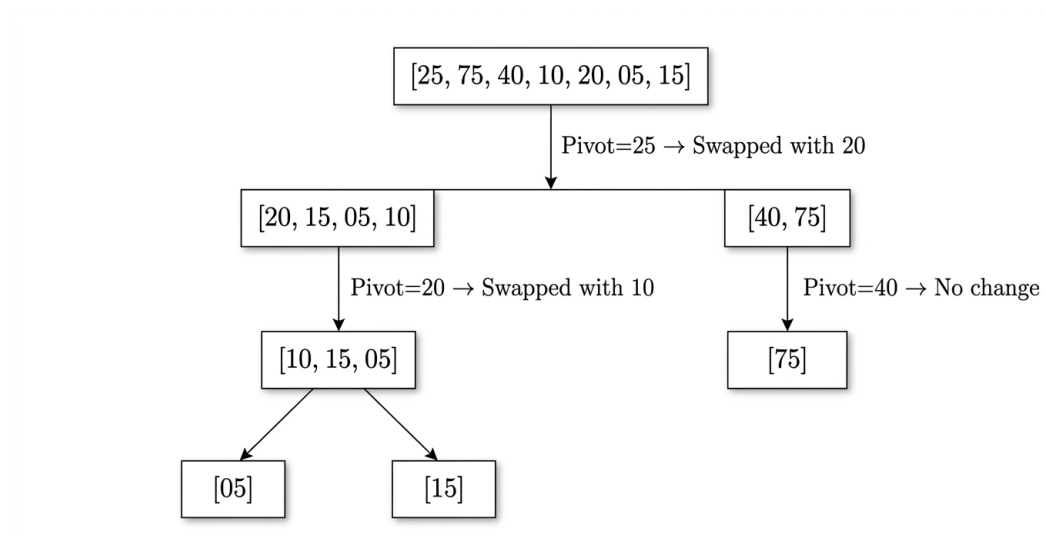


Figure 1: Tree Diagram of Quick Sort Trace.

This tree confirms the final sorted order.

3 Question: Explain Strassen's matrix multiplication and derive its time complexity. (10 Marks)

Introduction

Strassen's algorithm is a **divide-and-conquer** method for multiplying two $n \times n$ matrices that achieves better asymptotic efficiency than the standard $\Theta(n^3)$ definition-based multiplication. The key insight is that the product of two 2×2 matrices can be computed using only **7 multiplications** instead of the usual 8, at the cost of a few extra additions/subtractions.

1. The 2×2 Strassen Formulas

Consider two 2×2 matrices:

$$A = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix}, \quad B = \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix}$$

The product $C = A \times B$ is computed using the following **seven intermediate products**:

Product	Expression
m_1	$(a_{00} + a_{11}) \times (b_{00} + b_{11})$
m_2	$(a_{10} + a_{11}) \times b_{00}$
m_3	$a_{00} \times (b_{01} - b_{11})$
m_4	$a_{11} \times (b_{10} - b_{00})$
m_5	$(a_{00} + a_{01}) \times b_{11}$
m_6	$(a_{10} - a_{00}) \times (b_{00} + b_{01})$
m_7	$(a_{01} - a_{11}) \times (b_{10} + b_{11})$

The elements of the product matrix C are then obtained as linear combinations of these seven products:

$$c_{00} = m_1 + m_4 - m_5 + m_7$$

$$c_{01} = m_3 + m_5$$

$$c_{10} = m_2 + m_4$$

$$c_{11} = m_1 + m_3 - m_2 + m_6$$

This uses **7 multiplications** and **18 additions/subtractions** for the 2×2 case.

2. Recursive Generalization for $n \times n$ Matrices

For larger matrices (where n is a power of 2 for simplicity), the matrices are partitioned into four $n/2 \times n/2$ submatrices:

$$A = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix}, \quad B = \begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{bmatrix}$$

The same formulas are applied, but now the arithmetic operations (addition, subtraction, and multiplication) are performed on **submatrices** rather than scalars. The seven multiplications of submatrices are computed **recursively** using Strassen's algorithm itself, until the submatrices become 1×1 (base case).

3. Time Complexity Derivation

Let $M(n)$ be the number of scalar multiplications required to multiply two $n \times n$ matrices using Strassen's algorithm.

Recurrence for Multiplications

At each recursive step:

- The problem is divided into **7 subproblems** of size $n/2 \times n/2$.
- The cost of combining results involves only additions/subtractions, no extra multiplications.

Thus, the recurrence relation is:

$$M(n) = 7M\left(\frac{n}{2}\right) \quad \text{for } n > 1$$

with the base case:

$$M(1) = 1$$

Solving the Recurrence

Assume $n = 2^k$ (a power of 2). Then by backward substitution:

$$\begin{aligned} M(2^k) &= 7M(2^{k-1}) \\ &= 7[7M(2^{k-2})] = 7^2M(2^{k-2}) \\ &= 7^3M(2^{k-3}) \\ &\vdots \\ &= 7^kM(2^0) = 7^k \cdot 1 = 7^k \end{aligned}$$

Since $k = \log_2 n$, we have:

$$M(n) = 7^{\log_2 n}$$

Using the logarithmic identity $a^{\log_b c} = c^{\log_b a}$, this can be rewritten as:

$$M(n) = n^{\log_2 7}$$

The exponent $\log_2 7 \approx 2.807$. Therefore:

$$M(n) \in \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$$

Complexity of Additions

Let $A(n)$ be the number of scalar additions/subtractions. At each recursive step, there are **18 additions** of $n/2 \times n/2$ matrices. Each such addition involves $(n/2)^2$ scalar additions. The recurrence is:

$$A(n) = 7A\left(\frac{n}{2}\right) + 18\left(\frac{n}{2}\right)^2 \quad \text{for } n > 1, \quad A(1) = 0$$

By the **Master Theorem** (Case 1: $a = 7$, $b = 2$, $f(n) = \Theta(n^2)$; $n^{\log_b a} = n^{\log_2 7} \approx n^{2.807}$, and $f(n) = O(n^{\log_2 7 - \epsilon})$), the solution is:

$$A(n) \in \Theta(n^{\log_2 7})$$

4. Final Efficiency Class and Comparison

The overall time complexity of Strassen's algorithm is:

$$T(n) \in \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$$

Comparison with Standard Multiplication:

- **Brute-force:** $\Theta(n^3)$
- **Strassen:** $\Theta(n^{2.807})$

For large n , Strassen's algorithm performs significantly fewer multiplications. For example, when $n = 1024$:

$$n^{\log_2 7} \approx 1024^{2.807} \approx 2.8 \times 10^8 \quad \text{vs} \quad n^3 \approx 1.07 \times 10^9$$

representing roughly a factor of 4 improvement.

Note: While Strassen's algorithm has a better asymptotic growth rate, its constant factors are larger due to the overhead of matrix additions and recursion. Therefore, it is typically used only for very large matrices (e.g., $n > 100$), and even then, further optimized algorithms like Coppersmith–Winograd exist with even smaller exponents (the theoretical limit is unknown).

4 Question: Write an algorithm to sort numbers using Insertion Sort. Discuss its efficiency. (10 Marks)

Introduction

Insertion Sort is a simple and intuitive sorting technique based on the **decrease-by-one approach**, where elements are inserted into their correct position in a progressively sorted subarray.

Algorithm: Insertion Sort

The algorithm assumes that a smaller subarray is already sorted and inserts the next element into its correct position by scanning from **right to left**.

Algorithm 5 InsertionSort($A[0..n-1]$)

```

1: Input: Array  $A[0..n-1]$  of orderable elements
2: Output: Sorted array in nondecreasing order
3: for  $i \leftarrow 1$  to  $n-1$  do
4:    $v \leftarrow A[i]$ 
5:    $j \leftarrow i-1$ 
6:   while  $j \geq 0$  and  $A[j] > v$  do
7:      $A[j+1] \leftarrow A[j]$ 
8:      $j \leftarrow j-1$ 
9:   end while
10:   $A[j+1] \leftarrow v$ 
11: end for

```

Working Principle

- The left portion of the array is always **sorted**.
- Each new element is **inserted at the correct position** by shifting larger elements to the right.

Efficiency Analysis

The efficiency depends on the number of **key comparisons** ($A[j] > v$), and varies based on input order.

Worst Case (Strictly Decreasing Order)

- Maximum number of comparisons and shifts occur.

$$C_{\text{worst}}(n) = \frac{(n-1)n}{2} \in \Theta(n^2)$$

Best Case (Already Sorted Array)

- Only one comparison per iteration.

$$C_{\text{best}}(n) = n - 1 \in \Theta(n)$$

Average Case

- Approximately half of the worst-case comparisons:

$$C_{\text{avg}}(n) \approx \frac{n^2}{4} \in \Theta(n^2)$$

Practical Significance

Insertion Sort is particularly useful because:

- It performs **twice as fast on average** compared to its worst case
- It is highly efficient for **almost sorted arrays**
- It is simple and suitable for **small datasets**

Conclusion

Insertion Sort is an effective algorithm for small or nearly sorted inputs, with $\Theta(n)$ best-case performance and $\Theta(n^2)$ average and worst-case complexity.

5 Question: Design Quicksort Algorithm and Analyze Its Efficiency (10 Marks)

Design of Quicksort Algorithm

Quicksort is a sorting algorithm based on the **divide-and-conquer** strategy. Unlike merge sort (which divides by position), quicksort divides an array based on **value**.

Core Idea

1. **Divide:** Rearrange the array into two subarrays and a pivot element such that all elements in the left subarray are $\leq$ pivot, and all elements in the right subarray are $\geq$ pivot. The pivot is now in its final sorted position.
2. **Conquer:** Recursively sort the left and right subarrays.
3. **Combine:** No explicit work is needed; the array is sorted once the recursion finishes.

Pseudocode

Algorithm 6 Quicksort($A[l..r]$)

```

1: if  $l < r$  then
2:    $s \leftarrow \text{HOAREPARTITION}(A[l..r])$  ▷  $s$  is split position
3:   QUICKSORT( $A[l..s-1]$ )
4:   QUICKSORT( $A[s+1..r]$ )
5: end if

```

Algorithm 7 HoarePartition($A[l..r]$)

```

1:  $p \leftarrow A[l]$  ▷ Select pivot as the first element
2:  $i \leftarrow l$ 
3:  $j \leftarrow r + 1$ 
4: repeat
5:   repeat
6:      $i \leftarrow i + 1$ 
7:   until  $A[i] \geq p$ 
8:   repeat
9:      $j \leftarrow j - 1$ 
10:  until  $A[j] \leq p$ 
11:  SWAP( $A[i], A[j]$ )
12: until  $i \geq j$ 
13: SWAP( $A[i], A[j]$ ) ▷ Undo last swap when  $i \geq j$ 
14: SWAP( $A[l], A[j]$ ) ▷ Place pivot in correct position
15: return  $j$ 

```

Efficiency Analysis

The primary operation in Quicksort is **key comparison** during the partitioning process.

1. Best-Case Efficiency: $\Theta(n \log n)$

The best case occurs when the partitioning always splits the array **exactly in the middle** (i.e., the pivot is the median).

- Recurrence relation:

$$C_{\text{best}}(n) = 2C_{\text{best}}(n/2) + n \quad \text{for } n > 1, \quad C_{\text{best}}(1) = 0$$

- Solution (Master Theorem, Case 2: $a = 2, b = 2, f(n) = n$):

$$C_{\text{best}}(n) = n \log_2 n \in \Theta(n \log n)$$

2. Worst-Case Efficiency: $\Theta(n^2)$

The worst case occurs when the partitions are **skewed to the extreme** (one subarray is empty, the other has $n - 1$ elements). This happens if the array is already sorted (ascending or descending) and the first/last element is consistently chosen as the pivot.

- Recurrence relation:

$$C_{\text{worst}}(n) = C_{\text{worst}}(n - 1) + (n + 1) \quad \text{for } n > 1, \quad C_{\text{worst}}(1) = 0$$

- Solving by backward substitution:

$$C_{\text{worst}}(n) = (n + 1) + n + (n - 1) + \cdots + 3 = \frac{(n + 1)(n + 2)}{2} - 3$$

$$C_{\text{worst}}(n) \approx \frac{n^2}{2} \in \Theta(n^2)$$

3. Average-Case Efficiency: $\Theta(n \log n)$

Assuming all possible permutations of the input array are equally likely, the split position s can be anywhere from 0 to $n - 1$ with probability $1/n$.

- Recurrence relation:

$$C_{\text{avg}}(n) = \frac{1}{n} \sum_{s=0}^{n-1} [(n + 1) + C_{\text{avg}}(s) + C_{\text{avg}}(n - 1 - s)] \quad \text{for } n > 1$$

$$C_{\text{avg}}(0) = 0, \quad C_{\text{avg}}(1) = 0$$

- Solving the recurrence yields:

$$C_{\text{avg}}(n) \approx 2n \ln n \approx 1.39 n \log_2 n \in \Theta(n \log n)$$

Summary Table

Case	Occurrence Condition	Efficiency Class
Best	Splits exactly in the middle	$\Theta(n \log n)$
Worst	Already sorted array (skewed splits)	$\Theta(n^2)$
Average	Randomly ordered array	$\Theta(n \log n)$

Table 1: Quicksort efficiency summary.

Note

Despite a worst-case of $\Theta(n^2)$, Quicksort's average-case constant factor is smaller than mergesort and heapsort, making it the preferred choice for large random datasets in practice.

6 Question: Write Merge Sort algorithm and sort the list E, X, A, M, P, L, E. (8 Marks)

Introduction

Merge Sort is a classic **divide-and-conquer algorithm** that recursively divides the array into smaller parts, sorts them, and then merges them to produce a sorted sequence.

Algorithm: Merge Sort

1. Mergesort Procedure

ALGORITHM: Mergesort($A[0..n-1]$)

Algorithm 8 Mergesort

```

1: if  $n > 1$  then
2:    $mid \leftarrow \lceil n/2 \rceil$ 
3:    $B \leftarrow A[0..mid-1]$                                 ▷ Copy to left half
4:    $C \leftarrow A[mid..n-1]$                                 ▷ Copy to right half
5:   MERGESORT( $B$ )
6:   MERGESORT( $C$ )
7:   MERGE( $B, C, A$ )
8: end if

```

2. Merge Procedure

ALGORITHM: Merge($B[0..p-1], C[0..q-1], A[0..p+q-1]$)

Algorithm 9 Merge

```

1:  $i \leftarrow 0, j \leftarrow 0, k \leftarrow 0$ 
2: while  $i < p$  and  $j < q$  do
3:   if  $B[i] \leq C[j]$  then
4:      $A[k] \leftarrow B[i]$ 
5:      $i \leftarrow i + 1$ 
6:   else
7:      $A[k] \leftarrow C[j]$ 
8:      $j \leftarrow j + 1$ 
9:   end if
10:   $k \leftarrow k + 1$ 
11: end while
12: while  $i < p$  do
13:   $A[k] \leftarrow B[i]$ 
14:   $i \leftarrow i + 1, k \leftarrow k + 1$ 
15: end while
16: while  $j < q$  do
17:   $A[k] \leftarrow C[j]$ 
18:   $j \leftarrow j + 1, k \leftarrow k + 1$ 
19: end while

```

Sorting the List: E, X, A, M, P, L, E

Step 1: Initial Split

- Left: $[E, X, A, M]$
- Right: $[P, L, E]$

Step 2: Further Division

- $[E, X, A, M] \rightarrow [E, X]$ and $[A, M]$
 - $[E, X] \rightarrow [E], [X]$
 - $[A, M] \rightarrow [A], [M]$
- $[P, L, E] \rightarrow [P]$ and $[L, E]$
 - $[L, E] \rightarrow [L], [E]$

Step 3: Merging Small Arrays

- $[E] + [X] \rightarrow [E, X]$
- $[A] + [M] \rightarrow [A, M]$
- $[L] + [E] \rightarrow [E, L]$

Step 4: Merging Larger Arrays

- $[E, X] + [A, M] \rightarrow [A, E, M, X]$
- $[P] + [E, L] \rightarrow [E, L, P]$

Step 5: Final Merge

Merge $[A, E, M, X]$ and $[E, L, P]$:

```
Compare A and E -> A
Compare E and E -> E
Compare M and E -> E
Compare M and L -> L
Compare M and P -> M
Compare X and P -> P
Remaining: X
```

Result: $[A, E, E, L, M, P, X]$

Final Sorted List

A, E, E, L, M, P, X

Conclusion

Merge Sort efficiently sorts the list using recursive division and merging, achieving a time complexity of:

$$\Theta(n \log n)$$

In the worst case, it performs approximately:

$$n \log_2 n - n + 1$$

comparisons, making it highly efficient for large datasets.

7 Question: Apply the DFS-based algorithm to solve the Topological Sorting problem. (6 Marks)

Introduction

Topological sorting is used to order the vertices of a **directed acyclic graph (DAG)** such that for every directed edge $u \rightarrow v$, vertex u appears before v .

The **DFS-based approach** generates this order using the concept of finishing times.

DFS-Based Topological Sorting Algorithm

The method follows a structured recursive traversal:

1. **DFS Traversal:** Perform a **Depth-First Search (DFS)** on the given graph.
2. **Record Popping Order:** When a vertex becomes a **dead-end** (no unvisited outgoing edges), it is **popped off the recursion stack** and recorded.
3. **Reverse the Order:** The **reverse of the popping order** gives the topological order.
4. **Cycle Check:** If a **back edge** is encountered during DFS, the graph contains a cycle, and topological sorting is **not possible**.

Application on Given Graph

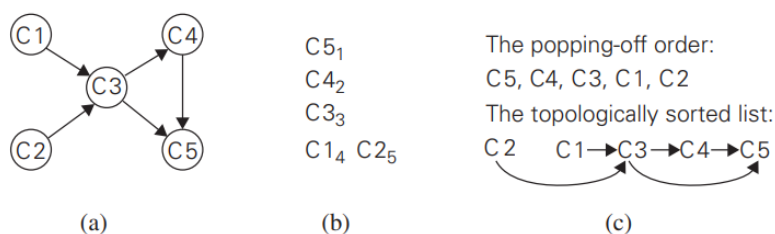


Figure 2: Graph for DFS-based topological sorting example.

Step 1: Perform DFS Traversal

- Traverse vertices and explore all descendants.
- When a vertex has no further unvisited nodes, it is **popped**.

Step 2: Popping-Off Order

The order in which vertices are removed from the stack:

C5, C4, C3, C1, C2

Step 3: Reverse the Order

Topological order is obtained by reversing:

C2, C1, C3, C4, C5

Verification

- In the obtained sequence, all edges go **from left to right**.
- Hence, all prerequisite constraints are satisfied.

Conclusion

The DFS-based method efficiently produces a valid topological ordering by reversing the vertex finishing order. Different valid orderings may exist, but all must satisfy the dependency constraints of the DAG.

8 Question: Write algorithms for Preorder, Inorder and Postorder traversals of a tree. Write traversals for the given tree. (6 Marks)

Introduction

Tree traversal algorithms visit every node of a binary tree. The three classic traversals—**Preorder, Inorder, and Postorder**—differ based on the **timing of visiting the root node**.

Traversal Algorithms

All traversals follow a recursive structure: if the tree is not empty, process the root and its subtrees.

1. Preorder Traversal (Root $\rightarrow$ Left $\rightarrow$ Right)

Algorithm 10 Preorder

```
1: procedure PREORDER( $T$ )
2:   if  $T$  is not empty then
3:     VISIT( $T$ .root)
4:     PREORDER( $T$ .left)
5:     PREORDER( $T$ .right)
6:   end if
7: end procedure
```

2. Inorder Traversal (Left $\rightarrow$ Root $\rightarrow$ Right)

Algorithm 11 Inorder

```
1: procedure INORDER( $T$ )
2:   if  $T$  is not empty then
3:     INORDER( $T$ .left)
4:     VISIT( $T$ .root)
5:     INORDER( $T$ .right)
6:   end if
7: end procedure
```

3. Postorder Traversal (Left $\rightarrow$ Right $\rightarrow$ Root)

Algorithm 12 Postorder

```

1: procedure POSTORDER(T)
2:   if T is not empty then
3:     POSTORDER(T.left)
4:     POSTORDER(T.right)
5:     VISIT(T.root)
6:   end if
7: end procedure
  
```

Traversals for the Given Tree

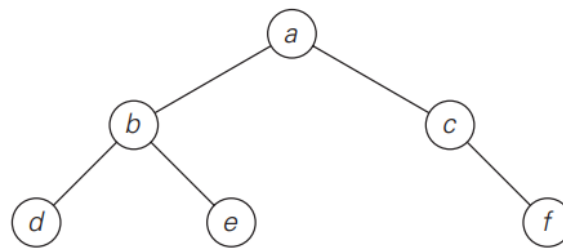


Figure 3: Binary tree used for traversal examples.

Given tree structure:

- Root: **a**
- Left child of a: **b**, Right child: **c**
- Children of b: **d** (left), **e** (right)
- Right child of c: **f**

Traversal Results

- **Preorder:** a, b, d, e, c, f
- **Inorder:** d, b, e, a, c, f
- **Postorder:** d, e, b, f, c, a

Additional Case (Extended Tree)

If an additional node **g** is added as a child of **d**:

- **Preorder:** a, b, d, g, e, c, f
- **Inorder:** d, g, b, e, a, f, c
- **Postorder:** g, d, e, b, f, c, a

Conclusion

All three traversals visit each node exactly once, resulting in $\Theta(n)$ time complexity. Each traversal serves different purposes depending on when the root is processed relative to its subtrees.

9 Question: What is Divide and Conquer? Develop Quick Sort algorithm and write its best case. Use it to sort: E, X, A, M, P, L, E. (10 Marks)

Introduction: Divide and Conquer

Divide-and-conquer is an algorithmic paradigm that solves a problem using a recursive three-step strategy:

1. **Divide:** Partition the problem into **smaller subproblems of the same type**.
2. **Conquer:** Solve the subproblems **recursively** (or directly if small).
3. **Combine:** Merge the solutions to obtain the final result.

Note: In **Quicksort**, combining is trivial since partitioning places elements in their correct position.

Quick Sort Algorithm

1. Main Algorithm: Quicksort($A[l..r]$)

- **Input:** Subarray $A[l..r]$
- **Output:** Sorted subarray

Algorithm 13 Quicksort

```
1: procedure QUICKSORT( $A, l, r$ )
2:   if  $l < r$  then
3:      $s \leftarrow \text{PARTITION}(A, l, r)$ 
4:     QUICKSORT( $A, l, s-1$ )
5:     QUICKSORT( $A, s+1, r$ )
6:   end if
7: end procedure
```

2. Partition Procedure (Hoare Method)

Algorithm 14 HoarePartition

```

1: procedure HOAREPARTITION(A, l, r)
2:    $p \leftarrow A[l]$ 
3:    $i \leftarrow l$ 
4:    $j \leftarrow r + 1$ 
5:   repeat
6:     repeat
7:        $i \leftarrow i + 1$ 
8:     until  $A[i] \geq p$ 
9:     repeat
10:       $j \leftarrow j - 1$ 
11:    until  $A[j] \leq p$ 
12:    if  $i < j$  then
13:      SWAP( $A[i]$ ,  $A[j]$ )
14:    end if
15:  until  $i \geq j$ 
16:  SWAP( $A[l]$ ,  $A[j]$ )
17:  return  $j$ 
18: end procedure

```

Best Case Efficiency

The best case occurs when the array is split into **two equal halves** at every step.

- **Recurrence:**

$$C_{\text{best}}(n) = 2C_{\text{best}}(n/2) + n, \quad C_{\text{best}}(1) = 0$$

- **Solution:**

$$C_{\text{best}}(n) = n \log_2 n$$

- **Efficiency Class:**

$$\Theta(n \log n)$$

Applying Quick Sort: E, X, A, M, P, L, E

Step 1: Initial Partition

- Pivot = E
- After partition:

$$[A, E, E, M, P, L, X]$$

Split position $s = 2$

Step 2: Left Subarray [A, E]

- Pivot = A
- Already sorted: $[A, E]$

Step 3: Right Subarray [M, P, L, X]

- Pivot = M
- After partition: $[L, M, P, X]$

Step 4: Final Subarrays

- $[L] \rightarrow$ sorted
- $[P, X] \rightarrow$ sorted

Final Sorted List

A, E, E, L, M, P, X

10 Question: Distinguish between Decrease & Conquer and Divide & Conquer with block diagram. Develop Insertion Sort algorithm and estimate efficiency. (10 Marks)

Introduction

Algorithm design techniques such as **Divide-and-Conquer** and **Decrease-and-Conquer** differ mainly in how they reduce problem size and construct the final solution.

Difference Between Divide & Conquer and Decrease & Conquer

Divide-and-Conquer

- Breaks a problem of size n into **multiple smaller subproblems** (typically $n/2$)
- Solves each subproblem recursively
- Requires a **combine step** to merge solutions

Block Diagram Concept:

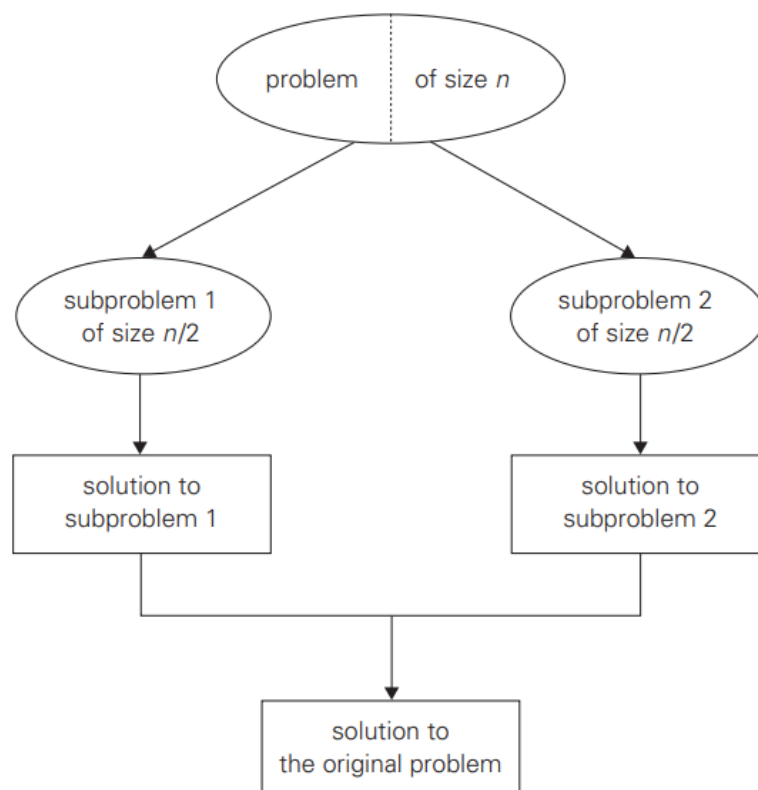


Figure 4: Block diagram for Divide-and-Conquer approach.

Decrease-and-Conquer

- Reduces problem to **one smaller subproblem** (e.g., $n - 1$)

- Solves the smaller problem
- **Extends** the solution to the original problem
- No combining of multiple solutions

Block Diagram Concept:

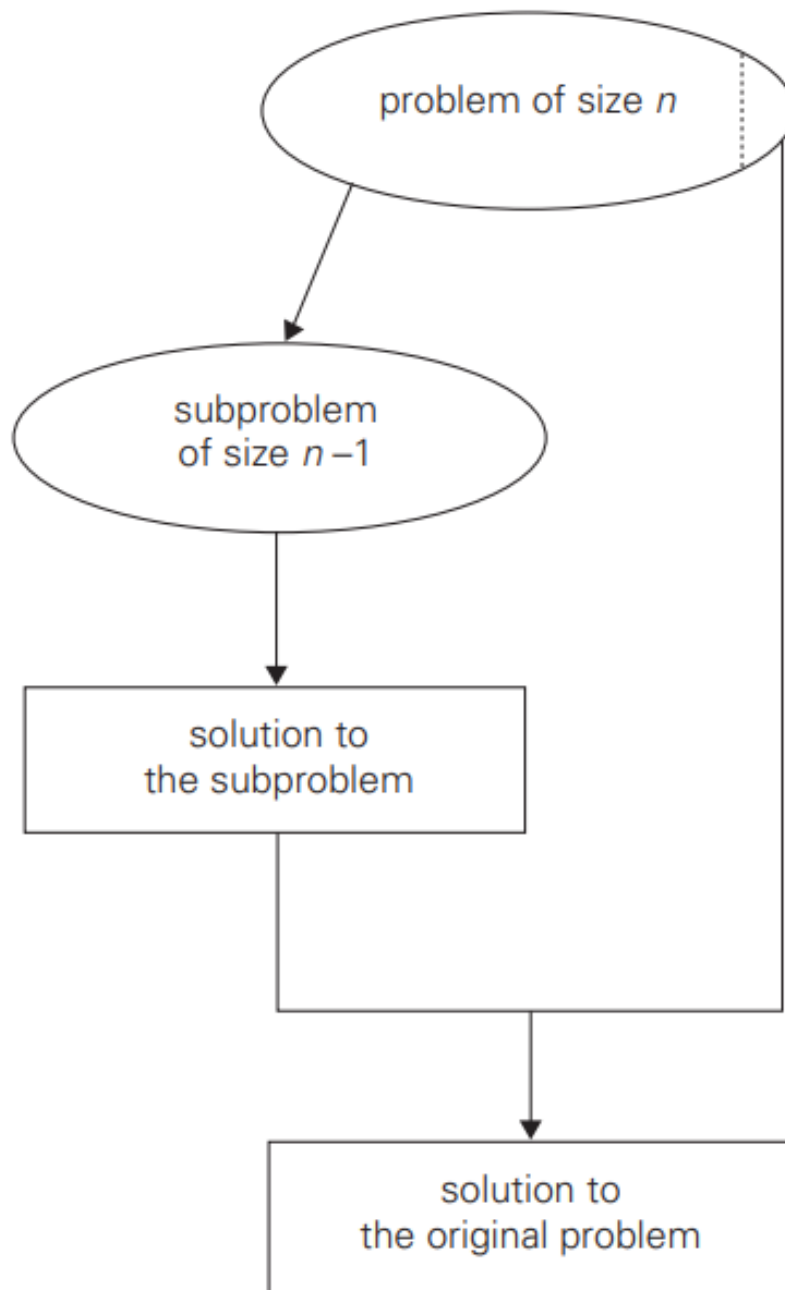


Figure 5: Block diagram for Decrease-and-Conquer approach.

Key Differences

Feature	Divide & Conquer	Decrease & Conquer
Number of Subproblems	Multiple	One
Combine Step	Required	Not required
Approach	Split & Merge	Reduce & Extend

Insertion Sort Algorithm**Introduction**

Insertion Sort is based on the **decrease-by-one technique**, where a sorted subarray is extended by inserting one element at a time into its correct position.

Algorithm: InsertionSort(A[0..n-1])

Algorithm 15 InsertionSort

```

1: for  $i \leftarrow 1$  to  $n - 1$  do
2:    $v \leftarrow A[i]$ 
3:    $j \leftarrow i - 1$ 
4:   while  $j \geq 0$  and  $A[j] > v$  do
5:      $A[j + 1] \leftarrow A[j]$ 
6:      $j \leftarrow j - 1$ 
7:   end while
8:    $A[j + 1] \leftarrow v$ 
9: end for

```

Efficiency Analysis

The efficiency depends on the number of **key comparisons** ($A[j] > v$).

Worst Case (Decreasing Order)

$$C_{\text{worst}}(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \in \Theta(n^2)$$

Best Case (Already Sorted)

$$C_{\text{best}}(n) = n - 1 \in \Theta(n)$$

Average Case

$$C_{\text{avg}}(n) \approx \frac{n^2}{4} \in \Theta(n^2)$$

11 Question: Define Topological Sorting. List the two approaches and illustrate with examples. (10 Marks)

Introduction / Definition

Topological sorting is defined for a **directed acyclic graph (DAG)** as a linear ordering of vertices such that for every directed edge $u \rightarrow v$, the **starting vertex** u appears before the **ending vertex** v .

- A topological order exists **only if the graph has no cycles**
- If a **back edge** is found (during DFS), the graph is **not a DAG**, hence sorting is not possible

Approaches to Topological Sorting

1. DFS-Based Algorithm

This method uses **Depth-First Search (DFS)**.

Method:

1. Perform DFS traversal of the graph
2. Record vertices when they become **dead-ends** (i.e., popped off the recursion stack)
3. Reverse this order to obtain the topological sort

Example: Given graph with edges:

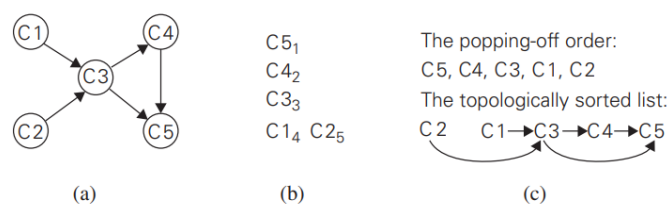


Figure 6: Graph for DFS-based topological sorting example.

- **Popping order:** $C5, C4, C3, C1, C2$
- **Topological order (reverse):** $C2, C1, C3, C4, C5$

2. Source-Removal Algorithm (Decrease-and-Conquer)

This method repeatedly removes vertices with no incoming edges.

Method:

1. Identify a **source vertex** (no incoming edges)
2. Remove it along with its outgoing edges

3. Repeat until all vertices are removed
4. The order of removal gives the topological order

If no source exists in a non-empty graph $\rightarrow$ cycle exists.

Example: Using the same graph:

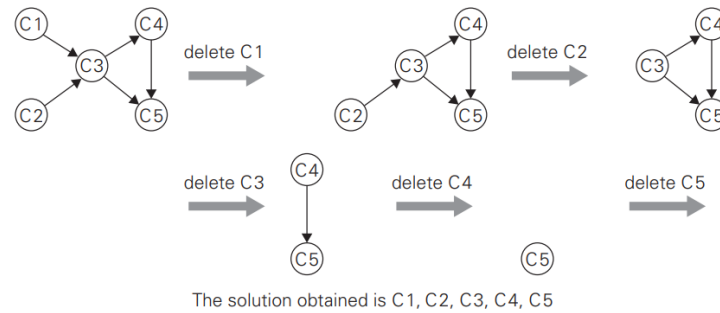


Figure 7: Source-removal process on the same DAG.

- Initial sources: **C1, C2**
- Remove in order: $C1 \rightarrow C2 \rightarrow C3 \rightarrow C4 \rightarrow C5$
- **Topological order: C1, C2, C3, C4, C5**

Key Observation

Multiple valid topological orders may exist. Example:

- C2, C1, C3, C4, C5
- C1, C2, C3, C4, C5

Practical Significance

Topological sorting is widely used in:

- **Instruction scheduling** in compilers
- **Dependency resolution** in linkers
- **Task scheduling** and project planning
- **Spreadsheet evaluation ordering**

12 Question: Apply Quick Sort to sort: P, R, O, G, R, A, M, M, I, N, G. Draw the tree of recursive calls. (10 Marks)

Quicksort Partitioning (Hoare's Method)

The algorithm uses the first element as a **pivot** ($p = A[l]$) and scans from both ends.

- **Left-to-right scan** (i): Stops at the first element $\geq p$.
- **Right-to-left scan** (j): Stops at the first element $\leq p$.
- **Partitioning Step**: When i and j stop, if they haven't crossed ($i < j$), elements are swapped. Once they cross ($i \geq j$), the pivot is swapped with $A[j]$, placing it in its final sorted position.

Step-by-Step Trace

Initial List

$[P, R, O, G, R, A, M, M, I, N, G]$

Pivot = P (index 0)

1. First Partition (Full list, l=0, r=10):

- i stops at 'R' (index 1), j stops at 'G' (index 10). Swap: $[P, G, O, G, R, A, M, M, I, N, R]$
- Next i stops at 'R' (index 4), j stops at 'N' (index 9). Swap: $[P, G, O, G, N, A, M, M, I, R, R]$
- Next i stops at 'R' (index 9), j stops at 'I' (index 8). Indices crossed ($i > j$).
- Swap pivot P with $A[j]$ ($A[8]='I'$): $[I, G, O, G, N, A, M, M, P, R, R]$
- Split position $s = 8$.

2. Left Subarray (l=0, r=7): $[I, G, O, G, N, A, M, M]$, Pivot = I

- i stops at 'O' (index 2), j stops at 'A' (index 5). Swap: $[I, G, A, G, N, O, M, M]$
- Next i stops at 'N' (index 4), j stops at 'G' (index 3). Indices crossed.
- Swap pivot I with $A[j]$ ($A[3]='G'$): $[G, G, A, I, N, O, M, M]$
- Split position $s = 3$.

3. Further Subdivisions:

- Left of $s = 3$ (l=0, r=2): $[G, G, A]$, Pivot = G . Split at $s = 1$. Result: $[A, G, G]$
- Right of $s = 3$ (l=4, r=7): $[N, O, M, M]$, Pivot = N . Split at $s = 6$. Result: $[M, M, N, O]$
- Right of $s = 8$ (l=9, r=10): $[R, R]$, Pivot = R . Split at $s = 10$. Result: $[R, R]$

Final Sorted List

$A, G, G, I, M, M, N, O, P, R, R$

Tree of Recursive Calls

The tree shows subarray bounds (l, r) and the obtained split position s for each call.

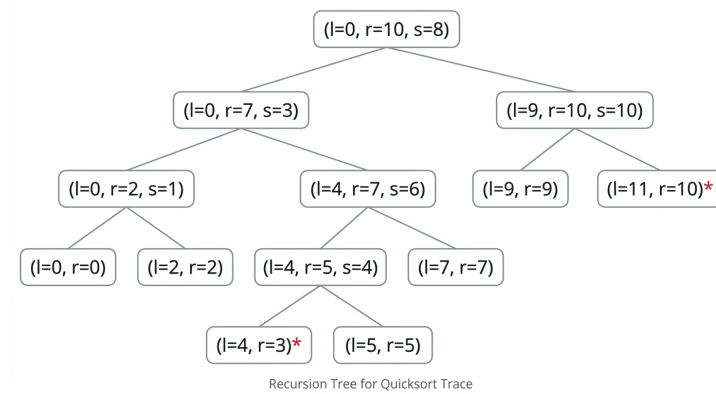


Figure 8: Recursion tree for Quick Sort on the given list. Subarrays with $l > r$ are empty base cases.

Efficiency Note

The number of key comparisons in the best case follows the recurrence $C_{\text{best}}(n) = 2C_{\text{best}}(n/2) + n$, resulting in $\Theta(n \log_2 n)$. On average, Quicksort makes approximately 39% more comparisons than the best case, and its inner loop is highly efficient. However, in the worst case (such as an already sorted list), the complexity can reach $\Theta(n^2)$.

13 Question: Apply Merge Sort and Quick Sort on: V, T, U, B, E, L, A, G, A, V, I. Draw tree of recursive calls and merge operations. (10 Marks)

Merge Sort Application

Input: V, T, U, B, E, L, A, G, A, V, I ($n = 11$)

Tree of Recursive Calls (Splitting)

The algorithm recursively divides the array until subarrays of size 1 are reached.

1. **Level 0:** [V, T, U, B, E, L, A, G, A, V, I]
2. **Level 1:** Divide into [V, T, U, B, E] and [L, A, G, A, V, I]
3. **Level 2:**
 - Left: [V, T] and [U, B, E]
 - Right: [L, A, G] and [A, V, I]
4. **Level 3:**
 - From [V, T] : [V] and [T]
 - From [U, B, E] : [U] and [B, E]
 - From [L, A, G] : [L] and [A, G]
 - From [A, V, I] : [A] and [V, I]
5. **Level 4 (Final Base Cases):**
 - From [B, E] : [B] and [E]
 - From [A, G] : [A] and [G]
 - From [V, I] : [V] and [I]

Figure 9: Tree of Recursive Calls (Splitting).

Tree of Merge Operations (Combining)

Merging in non-decreasing alphabetical order:

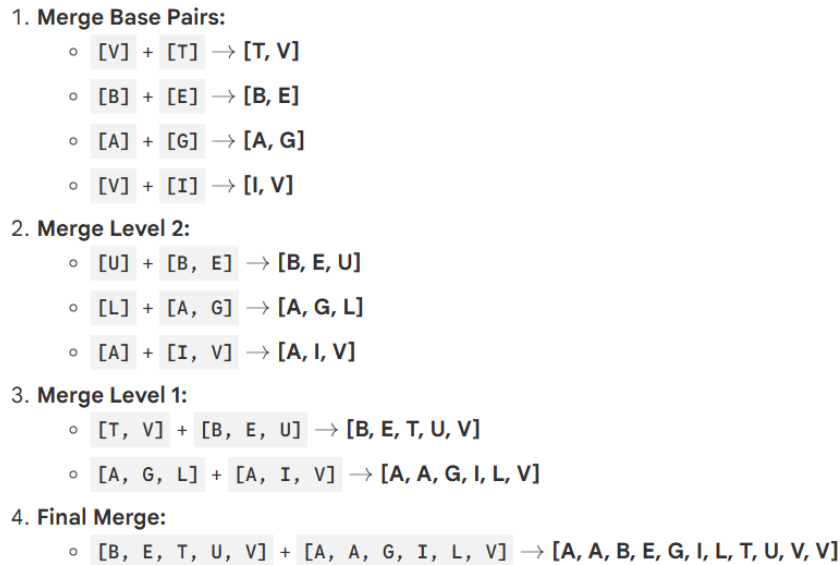


Figure 10: Tree of Merge Operations (Combining).

Final Sorted List (Merge Sort):

$A, A, B, E, G, I, L, T, U, V, V$

Quick Sort Application**Initial list:** $[V, T, U, B, E, L, A, G, A, V, I]$ **First Partition (Hoare's method, pivot = first element 'V')**

- Pivot $p = V$ (index 0).
- Left scan i : stops at index 9 ($V \geq V$).
- Right scan j : stops at index 10 ($I \leq V$).
- Swap $A[i]$ and $A[j]$: $[V, T, U, B, E, L, A, G, A, I, V]$.
- Continue: i moves to index 10 ($V \geq V$), j moves to index 9 ($I \leq V$). Indices crossed.
- Swap pivot $A[l]$ with $A[j]$: swap index 0 ('V') with index 9 ('I').
- Result: $[I, T, U, B, E, L, A, G, A, V, V]$ with split position $s = 9$.

Recursive Subproblems

- Left subarray ($l = 0, r = 8$): $[I, T, U, B, E, L, A, G, A]$
- Right subarray ($l = 10, r = 10$): $[V]$ (already sorted)

The process continues recursively on the left subarray, eventually producing the sorted order.

Final Sorted List (Quick Sort)

$A, A, B, E, G, I, L, T, U, V, V$

Efficiency Summary

- **Merge Sort:** Consistent $\Theta(n \log n)$ performance, requires $\Theta(n)$ extra space.
- **Quick Sort:** Average $\Theta(n \log n)$, worst-case $\Theta(n^2)$, in-place partitioning, usually faster in practice.

14 Question: Apply Strassen's Algorithm for matrix multiplication and justify why it is better. (10 Marks)

Applying Strassen's Algorithm

Strassen's algorithm is an application of the **divide-and-conquer** technique that improves the asymptotic efficiency of matrix multiplication. The algorithm is based on the discovery that two 2×2 matrices can be multiplied using only **seven multiplications** instead of the eight required by the standard brute-force method.

To multiply two $n \times n$ matrices A and B (where n is a power of 2), the matrices are first divided into **four** $n/2 \times n/2$ **submatrices**:

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} \cdot \begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{bmatrix}$$

These submatrices are treated as numbers in the following formulas to compute seven intermediate products, M_1 through M_7 , recursively:

$$M_1 = (A_{00} + A_{11}) \cdot (B_{00} + B_{11})$$

$$M_2 = (A_{10} + A_{11}) \cdot B_{00}$$

$$M_3 = A_{00} \cdot (B_{01} - B_{11})$$

$$M_4 = A_{11} \cdot (B_{10} - B_{00})$$

$$M_5 = (A_{00} + A_{01}) \cdot B_{11}$$

$$M_6 = (A_{10} - A_{00}) \cdot (B_{00} + B_{01})$$

$$M_7 = (A_{01} - A_{11}) \cdot (B_{10} + B_{11})$$

The final submatrices of the product matrix C are obtained by combining these products with 18 matrix additions and subtractions:

$$C_{00} = M_1 + M_4 - M_5 + M_7$$

$$C_{01} = M_3 + M_5$$

$$C_{10} = M_2 + M_4$$

$$C_{11} = M_1 + M_3 - M_2 + M_6$$

Justification: Why It Is Better

Strassen's algorithm is considered "better" because of its **asymptotic superiority** as the matrix order n grows large.

1. **Reduced Multiplications:** The brute-force algorithm requires $\Theta(n^3)$ multiplications. Strassen's algorithm follows the recurrence:

$$M(n) = 7M(n/2), \quad M(1) = 1$$

Solving using the Master Theorem gives:

$$M(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$$

Since $2.807 < 3$, Strassen's algorithm performs significantly fewer multiplications for large n .

2. **Additions Complexity:** Although the algorithm introduces extra additions and subtractions, the recurrence for additions is:

$$A(n) = 7A(n/2) + 18 \left(\frac{n}{2}\right)^2, \quad A(1) = 0$$

Using the Master Theorem (Case 1, because n^2 grows slower than $n^{\log_2 7}$), the number of additions also lies in $\Theta(n^{\log_2 7})$.

3. **Overall Time Efficiency:** Because both multiplications and additions grow at a rate lower than $\Theta(n^3)$, Strassen's algorithm achieves a **better efficiency class** than the conventional method. For large matrices, this asymptotic advantage translates into faster execution despite higher constant factors.

Thus, Strassen's algorithm is asymptotically superior and is preferred for large-scale matrix multiplication.

15 Question: Write the control abstraction for Divide and Conquer technique. (4 Marks)

Control Abstraction: Divide and Conquer

Divide-and-Conquer is a recursive strategy that solves a problem by breaking it into smaller instances, solving them, and combining their results.

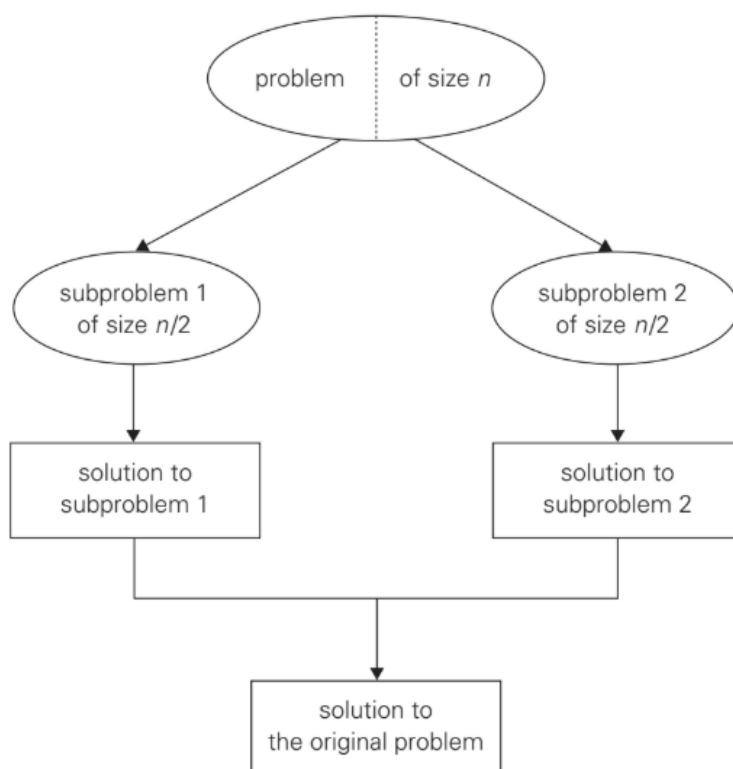


Figure 11: Divide-and-conquer technique.

General Control Abstraction

Algorithm 16 Divide_Conquer(P)

```

1: if problem size is small then
2:   Solve directly
3: else
4:   Divide: Partition problem  $P$  into smaller subproblems
5:   Conquer: Recursively solve each subproblem
6:   Combine: Merge the solutions to obtain the final result
7: end if
  
```

Explanation

- **Divide:** Break problem into smaller parts (same type)
- **Conquer:** Solve subproblems recursively until base case

- **Combine:** Construct final solution from sub-solutions

Conclusion

This abstraction provides a general template for designing recursive algorithms like **Merge Sort, Quick Sort, and Strassen's algorithm**, where problem-solving is structured into divide, conquer, and combine phases.

16 Question: Divide and Conquer (4M); Stable Algorithms & Quicksort (6M); Sorting by Position vs Value (10M)

1. Divide and Conquer (4 Marks)

Definition

Divide-and-conquer is an algorithmic technique that solves a problem using a recursive three-step strategy:

1. **Divide:** Break the problem into **smaller subproblems of the same type**
2. **Conquer:** Solve subproblems **recursively** (or directly if small)
3. **Combine:** Merge subproblem solutions to obtain the final result

Example: Merge Sort divides an array into halves, sorts them, and merges them.

2. Stable Algorithms and Quicksort (6 Marks)

Definition of Stability

A sorting algorithm is **stable** if it **preserves the relative order of equal elements** in the sorted output.

- If two elements have the same key, their **original order remains unchanged**

Is Quicksort Stable?

- **Quicksort is NOT stable**

Reason: During partitioning, elements may be swapped across positions, disturbing the original order of equal elements.

Key Insight

- **Stable example:** Merge Sort
- **Unstable example:** Quick Sort

3. Sorting by Position vs Sorting by Value (10 Marks)

The distinction is based on how the input is divided.

Sorting by Position (Merge Sort)

- Division based on **array indices (positions)**.
- **Process:** Split array into halves: $A[0..\lfloor n/2 \rfloor - 1]$ and $A[\lfloor n/2 \rfloor..n - 1]$.
- **Key Characteristics:**

- Division is **simple and immediate**
- **Major work happens in merging stage**
- Requires **extra space**
- Complexity: $\Theta(n \log n)$

Sorting by Value (Quick Sort)

- Division based on **element values**.
- **Process:** Choose a **pivot**, partition elements: Left $\leq$ pivot, Right $\geq$ pivot.
- **Key Characteristics:**
 - **Major work happens in partitioning stage**
 - Pivot is placed in its **final sorted position**
 - No explicit combine step required
 - Complexity: Average $\Theta(n \log n)$, Worst $\Theta(n^2)$

Comparison Summary

Feature	Merge Sort (Position)	Quick Sort (Value)
Division	By index	By pivot value
Main Work	Combine (merge)	Divide (partition)
Stability	Stable	Not stable
Complexity	$\Theta(n \log n)$	Avg: $\Theta(n \log n)$, Worst: $\Theta(n^2)$
Space	Extra memory required	In-place

Table 2: Comparison of sorting by position vs sorting by value.

Conclusion

Divide-and-conquer provides a powerful framework for algorithm design. Stability is crucial when order matters, and quicksort, though efficient, is **not stable**. The distinction between **position-based** and **value-based** sorting highlights fundamental differences in how algorithms operate and distribute computational work.